

Técnicas de los Sistemas Inteligentes

Enunciado de la Práctica 3

Introducción

La tercera práctica de la asignatura de *Técnicas de los Sistemas Inteligentes* consiste en diseñar e implementar un dominio de planificación clásico siguiendo las indicaciones que se dan en este enunciado y en la presentación de la práctica. Para el desarrollo de la práctica los estudiantes deberán confeccionar un conjunto de ficheros PDDL, y ejecutarlos en el planificador [Fast Downward](#) para obtener un plan óptimo en número de acciones.

Descripción del mundo

Nos movemos en la Tierra Media de El Señor de los Anillos¹, en donde hay una serie de localizaciones y caminos entre estas. Las localizaciones que se emplearán en la práctica, así como las vías de comunicación entre dichas localizaciones, se muestra en la siguiente figura.



¹ https://en.wikipedia.org/wiki/The_Lord_of_the_Rings



En concreto, disponemos de las siguientes localizaciones: Hobbiton, Bree, Rivendell, HighPass, Mirkwood, Erebor, Moria, Lothlorien, Tharbad, Fangorn, Isengard, Helm's Deep, Edoras, Amon Hen, Minas Tirith, Dol Amroth, Tolfolas, Minas Morgul, Dead Marshes y Orodruin (también conocido como el Monte del Destino). Debemos ser conscientes de que, cuando se transita el camino que une Rivendell con Moria (da igual si es de Rivendell a Moria o de Moria a Rivendell), dicho camino se destruye (y no puede volver a ser utilizado), dado que el Guardián del Agua (https://en.wikipedia.org/wiki/Watcher_in_the_Water) acecha la entrada a Moria y destruye el camino si alguien osa tratar de transitarlo.

Dentro de este mapa de la Tierra Media, tenemos nodos de Minerales en:

- Moria: Famosa por sus vastas minas. Moria también es un nodo de Mithril, metal extremadamente valioso y ligero.
- Erebor: Conocida por sus minas de oro y piedras preciosas, fue el reino de los enanos antes de ser ocupado por el dragón Smaug.

También disponemos de nodos de Madera en:

- Fangorn: Uno de los bosques más antiguos y densos de la Tierra Media, hogar de los *ents*. Rico en robles y pinos.
- Lothlorien: Bosque élfico con árboles *mallorn*, de madera plateada y hojas doradas.
- Mirkwood: Gran extensión forestal al este del río Anduin, con abundante vegetación.

Finalmente, se dispone de Alimentos en Hobbiton y de Especies en Tolfolas.

Los distintos tipos de Personaje están especializados a la hora de extraer recursos. Los personajes necesarios para cada ejercicio se irán presentando e introduciendo en cada enunciado, pero, a modo de resumen, se proporciona el siguiente listado de tipos de Personaje y recursos que pueden extraer:

- Enano: Madera, Mineral, Mithril
- Hobbit: Alimento
- Orco: Mineral y Madera
- Humano: Madera
- Corsario: Especia

Descripción de los ejercicios a realizar

Los estudiantes deberán definir un dominio de planificación escrito en PDDL que represente el mundo presentado en la sección anterior y, a medida que se realicen los ejercicios propuestos, ir añadiéndole nuevas funcionalidades.

Notas de aplicación general a toda la práctica:

- Es **imprescindible respetar tanto el nombre de las acciones como el orden de los parámetros** para que el plan generado de vuestro dominio pueda ser comparado con la solución de la práctica. Del mismo modo, está terminantemente **prohibido crear acciones nuevas**, al margen de las indicadas en el enunciado.
- Salvo que se indique lo contrario, entre ejercicios no se pueden eliminar acciones, aunque estas, en un principio, puedan parecer innecesarias para un determinado ejercicio. Es decir, **todas las acciones anteriores se deben conservar**. La idea es construir de forma incremental los distintos ejercicios, dejando incluso los comentarios de código anteriores. Así, si en un ejercicio se quiere leer o analizar el código de un apartado anterior no es necesario buscar retrospectivamente en otros ejercicios los comentarios de la acción o predicado que se quiera revisar.
- Todos los ejercicios deben ejecutarse con el siguiente alias:



```
./fast-downward.py --alias seq-opt-lmcut dominio.pddl problema.pddl
```

El único ejercicio que se debe ejecutar con un alias diferente es el Ejercicio 4. En ese caso, se debe emplear:

```
./fast-downward.py --alias seq-sat-lama-2011 --search-time-limit 200s dominio.pddl problema.pddl
```

Ejercicio 1: viajar para desplazarse y trabajar para extraer recursos (2 puntos)

En este ejercicio se pretende diseñar y programar una versión inicial del dominio de planificación propuesto, que servirá de base al resto de ejercicios. Se debe cumplir con los siguientes requisitos:

- Representar en el dominio, al menos, los **tipos: Personaje, Recurso, y Localización**. Usar **constantes** para representar el *tipo de personaje* **Enano** y **Hobbit**, y los *tipos de recurso* **Mineral, Mithril, Madera, Especia y Alimento**.
- Definir al menos los **predicados** necesarios para:
 - Determinar si un personaje o nodo de recurso concreto está en una localización determinada.
 - Representar que existe un camino entre dos localizaciones.
 - Determinar si un personaje está trabajando en una localización extrayendo un recurso.
 - Identificar si un personaje está disponible (es decir, no está trabajando).
 - Determinar que un tipo de personaje puede extraer un recurso.

En caso de que el estudiante considere que necesita más tipos, constantes o predicados para realizar correctamente el ejercicio, tiene libertad total para crearlos, pero debe justificarlo y explicarlo con suficiente detalle tanto en comentarios en el código como en la memoria.

- El dominio debe contener **únicamente** las siguientes dos **acciones**:
 - **Viajar**: Mueve un personaje entre dos localizaciones.
 - Parámetros: Personaje, Localización origen, Localización destino
 - Es importante remarcar que, en un primer momento, el hecho de ir de una ubicación a otra adyacente tiene coste unitario. En otras palabras, ir de Tharbad a Helm's Deep tiene el mismo coste/distancia que ir de Helm's Deep a Isengard.
 - **ExtraerRecurso**: Asigna un personaje a un nodo de recurso.
 - Parámetros: Personaje, Localización del recurso, Tipo de recurso
 - Solo ciertos tipos de personajes pueden extraer algunos recursos, como se puede comprobar en el listado de más arriba.
- El **problema** debe definir el mapa de localizaciones expuesto anteriormente. Tenemos 2 Enanos y 1 Hobbit. El Enano1 se encuentra en Tharbad visitando a su primo, mientras que el Hobbit1 se encuentra en Lothlorien, y el Enano2 se encuentra en Isengard. El Enano1 y el Hobbit1 están perfectamente disponibles, pero el Enano2 no lo está (dado que se encuentra ocupado con una misión diplomática).
- El **objetivo** (:goal) de este ejercicio es generar recursos de tipo Mithril y Alimento (o, en otras palabras, que haya un personaje asignado a un nodo/localización en donde haya dichos recursos).

A nivel teórico-conceptual, el alumnado debe explorar el fichero output.sas, explicar su contenido y posible utilidad, y revisar y describir en detalle las instancias creadas del operador "extraerrecurso" (indicando qué objetos de tipo personaje pueden extraer recursos y en qué ubicaciones).



Ejercicio 2: formar Comunidad reducida y destruir el anillo único (2 puntos)

Ahora, partiendo del ejercicio anterior, debemos formar la Comunidad del Anillo, recoger dicho anillo y portarlo al Monte del Destino (Orodruin), donde será destruido². Para ello, hay una serie de etapas que se deben cumplir:

- En primer lugar, en este ejercicio se emplea una Comunidad reducida, formada por 1 Hobbit, y 1 Mago. Esto implica que habría que expandir los tipos de personaje de nuestro dominio (para incluir la existencia de magos). Luego, para formar la comunidad, todos deben reunirse en un punto de la geografía (que debe ser escogido automáticamente por el *planner* con el objetivo de minimizar el número de acciones) y, en el momento en que estén todos juntos, la comunidad quedará formada. En concreto, tenemos Hobbit1, Hobbit2, Hobbit3 en Hobbiton; tenemos Hobbit4 en Bree; y tenemos Mago1 en Rivendell y Mago2 en Isengard. También tenemos un Enano1 disponible en Moria, y un Enano2 que sigue en misión diplomática en Isengard y, como consecuencia de ello, sigue sin estar disponible. A continuación, se debe crear:
 - Una nueva acción llamada **formarComunidad**, cuyos parámetros serían los 2 personajes que constituyen la Comunidad, y el lugar en donde se ha constituido.
 - Es importante remarcar que, en cuanto la Comunidad se forma, todos se desplazan conjuntamente. Es decir, si uno de los miembros de la Comunidad va a Isengard, todos deben ir a Isengard. Por ello, también deberíamos crear una acción denominada **viajarComunidad**, cuyos parámetros serían, por este orden, todos los personajes que se desplazan conjuntamente, la localización de origen, y la de destino.
 - Esto implica que debemos modificar también la acción **Viajar** (definida en el Ejercicio 1), dado que un personaje solo puede viajar (de forma individual) si la comunidad no está formada o, en caso de estar formada, si el personaje no pertenece a ella.
- En segundo lugar, una vez formada la Comunidad, deben recoger el anillo y, para poder recoger el anillo, es necesario que el personaje que lo recoja sea un Hobbit, que la Comunidad esté formada, que dicho Hobbit sea miembro de la Comunidad, y que esté en la localización en donde esté el anillo (el anillo se encuentra en Rivendell). A partir de ese momento, este Hobbit será el portador del anillo³. A continuación, la Comunidad debe recoger el Chaleco de Mithril en Moria (una vez ha sido creado por un Mago) y una Espada en Lothlorien. Y el portador del anillo, que debe ser el mismo del Chaleco de Mithril y de la Espada, debe llevar el anillo a Orodruin para destruirlo. Para ello, se deben crear las siguientes acciones:
 - **materializarChaleco**, cuyos parámetros serían el personaje de tipo Mago que lo crea y la ubicación (que debe ser una ubicación en donde se esté extrayendo Mithril). El mago que lo crea tiene que ser parte de la comunidad (por lo tanto, la comunidad ya debe estar formada para poder crear el chaleco);
 - **recogerObjeto**, cuyos parámetros serían el personaje que lo recoge, el lugar en donde se recoge, y el objeto a recoger. En concreto, como se indicaba antes, tras recoger el Anillo, se debe recoger (da igual en qué orden) una Espada y el Chaleco de Mithril;
 - **destruirAnillo**, cuyos parámetros serían el personaje que tira el Anillo y el lugar en donde se destruye. Es importante tener claro que, para poder coger el Chaleco de Mithril o la Espada, es imprescindible, primero, haber recogido el Anillo; y, una vez

² https://en.wikipedia.org/wiki/One_Ring

³ No es válido forzar a que un hobbit concreto sea el portador del anillo. Esa tarea le corresponde al planificador, y no se puede realizar a mano programáticamente.



se tienen los tres objetos (Anillo, Chaleco de Mithril, Espada⁴) ya se puede llevar a cabo la destrucción del Anillo. Para indicar el lugar en donde se destruye se debe crear un nuevo predicado **lugarDestruccion** que permita establecer, en el fichero de problema, dónde se debe destruir⁵.

El objetivo, por tanto, de este ejercicio, es destruir el Anillo Único. Es decir, `(:goal1 (anilloDestruido))`

Ejercicio 3: se amplía la Comunidad y observamos sus efectos (3 puntos)

En este ejercicio vamos a expandir el anterior con el objetivo de generar una tabla que nos muestre cómo evolucionan los tiempos de planificación, y la longitud de los planes, a medida que aumenta el tamaño de la Comunidad. Es decir, debemos cubrir una tabla como la que se muestra a continuación:

Tamaño de la Comunidad	Tiempo de Planificación (“Planner time”)	Longitud del Plan encontrado (“Plan length”)
(a): 1 hobbit y 1 mago		
(b): 2 hobbits y 1 mago		
(c): 3 hobbits y 1 mago		
(d): 3 hobbits, 1 mago y 1 elfo		

En este caso partimos, en el fichero de problema, de 4 Hobbits (3 ubicados en Hobbiton y 1 en Bree), 2 Magos (1 ubicado en Rivendell y otro en Isengard), 1 Elfo (ubicado en Lothlorien), 2 Enanos (ubicados en Moria y Fangorn), todos ellos disponibles. El objetivo sigue siendo destruir el anillo. En este caso, habrá que ir adaptando/modificando progresivamente el número de parámetros de las acciones **formarComunidad** y **viajarComunidad** para ajustarse a las dimensiones de la Comunidad que se solicita.

Se deben entregar todos los códigos implementados, empleando la siguiente nomenclatura: dominio3a.pddl, problema3a.pddl, dominio3b.pddl, problema3b.pddl, dominio3c.pddl, problema3c.pddl, dominio3d.pddl y problema3d.pddl.

Se deben discutir los resultados obtenidos y, en base a ellos, así como a la bibliografía recomendada y la literatura científica que el alumnado considere relevante, se debe profundizar en la clase de complejidad computacional de la planificación automática. ¿Cómo escala la resolución de los problemas a medida que incorporamos más acciones, objetos y/o nodos del mapa? Por ejemplo, ¿el planificador proporcionaría con rapidez un plan si nuestra Comunidad estuviese formada por 4 hobbits, 1 mago, 1 elfo, 1 enano y 2 humanos?

¿Es posible que, al igual en la Práctica 2, surjan soluciones simétricas? ¿De qué modo podrían aparecer estas y qué impacto tendrían en el rendimiento?

⁴ Estos tres objetos deben ser definidos como constantes, de nombre Anillo, ChalecoMithril y Espada.

⁵ Es importante remarcar también un principio de diseño en problemas de planificación automática. Lo ideal es desacoplar al máximo la lógica del dominio (fichero de dominio) de la tarea concreta que queremos abordar (fichero de problema). Por ejemplo, en lugar de incorporar de forma *hardcoded* en el fichero de dominio, como parte de la acción **destruirAnillo**, que la localización en donde debe estar el Hobbit es en Orodruin, generalmente será preferible indicarlo en el fichero de problema en un predicado creado para este propósito. De este modo si, por el motivo que sea, cambia la ubicación de destrucción del anillo, solo tendremos que modificarlo en el fichero de problema.



Ejercicio 4: crear Uruk-Hai en Torre de Hechicería (2 puntos)

En este ejercicio partimos del código desarrollado en el Ejercicio3a, y queremos crear una nueva raza de orco llamada Uruk-Hai y, para ello, previamente necesitamos construir una Torre de Hechicería en Isengard.

A su vez, para construir dicha torre, que debe ser creada de forma mágica por un Mago, necesitamos Mineral y Madera. Sabemos que estos recursos pueden ser extraídos por los Enanos, pero, como la Torre de Hechicería tiene un componente maligno, para su construcción dichos recursos deben ser extraídos forzosamente por Orcos (dado que los Enanos son muy orgullosos y nunca accederían a trabajar con este fin)⁶. Para extraer el Mineral, se necesita que un Humano construya en la misma localización (en la que se extrae Mineral) un Extractor (que, a su vez, necesita Madera, que puede ser extraída por Enanos, Humanos y Orcos). No obstante, de nuevo, los Enanos se niegan a extraer Madera que permita, luego, construir un Extractor que, a su vez, sea usado para construir la Torre de Hechicería.

Se asume que, en el momento en que un personaje está trabajando produciendo/generando algún recurso, ese recurso ya está accesible (en la cantidad necesaria) para la tarea en cuestión. Por ejemplo, en el momento en que un Enano o un Orco está trabajando en la extracción del Mineral, se asume que ya está a disposición de otro personaje para construir el Extractor y que la cantidad extraída ya es suficiente para construirlo. En el momento que se le asigne un nodo de recursos a un Personaje, se asume que ya queda allí trabajando todo el tiempo, proporcionando el recurso que sea. En otras palabras, si un Personaje está trabajando ya no vuelve a estar disponible. La única excepción es la construcción de un edificio (sea la Torre de Hechicería o el Extractor), dado que, en cuanto termina (es decir, en cuanto se ejecuta la acción), se asume que el trabajador, sea quien sea, vuelve a estar disponible.

Del mismo modo, necesitamos una acción **Construir**, que recibe, en este orden, el personaje que construye, el tipo de edificio a construir, y la localización donde se construye. El personaje debe estar en la localización en donde se vaya a construir el edificio (y en una misma ubicación solo puede haber, como máximo, un edificio de cada tipo). Dicha acción debe servir indistintamente tanto para construir una Torre de Hechicería (construida por un Mago, que no puede formar parte de la comunidad) como para construir un Extractor (construido por un Humano). Esta acción debe inferir por sí misma si se tiene el tipo de recurso necesario para poder ejecutarse. Es decir, no se le debe pasar el recurso concreto como parámetro a la acción, sino que la propia acción, sabiendo el tipo de recurso necesario para construir cada tipo de edificio, itera sobre los posibles recursos verificando si estos se están generando/extrayendo. Para ello, **se deben emplear forzosamente los comandos *exists* y *forall***, y un predicado que indique el recurso que necesita cada edificio.

Una vez se ha construido la Torre de Hechicería, se debe crear el Uruk-Hai (para lo cual sería necesario implementar una nueva acción llamada **CrearUrukHai**, a la que se le pasa el Mago que lo crea y la localización en donde se crea). Esta nueva raza de orcos es creada por un Mago (no perteneciente a la Comunidad), y necesita también Especia que debe ser extraída y transportada físicamente (desde Tolfolas a Isengard) por un Corsario (empleando la acción Viajar)⁷. Para recoger la Especia ya extraída es necesario crear una nueva acción denominada **RecogerEspecia**, que tiene como parámetros el corsario que la recoge y la ubicación donde la recoge (que es donde se extrae). Por lo tanto, el proceso de recogida y transporte de la Especia sería como sigue: un corsario la extrae

⁶ Es decir, los enanos, como tal, pueden extraer Mineral, Madera y Mithril, y así debe ser definido en el fichero de problema, pero es el propio planificador, al identificar que la Torre de Hechicería necesita Mineral, el que descarte para su extracción a los enanos. En otras palabras, cuando el Mineral, o cualquier otro recurso, vaya a ser usado en alguna tarea “maligna”, los enanos no podrán participar en su extracción.

⁷ Por lo tanto, sería necesario un Corsario para extraer la Especia y otro para Transportarla. Véase que, a diferencia de los otros recursos, la Especia no está accesible universalmente para realizar la tarea en cuestión, sino que debe ser transportada físicamente a su destino para su uso. El transporte se realiza por medio de la acción Viajar.



(empleando la acción `ExtraerRecurso`), otro corsario la recoge (empleando la acción `RecogerEspecia`), y este mismo corsario es el que la lleva a su destino (empleando la acción `Viajar`). No es necesario depositar la especia ni ejecutar ninguna acción especial para liberarla. Basta con llegar al destino.

En el momento en que la Torre esté construida y llegue la Especia a la Torre, ya se podría crear el Uruk-Hai.

En el fichero de este problema tendremos 4 Hobbits (3 ubicados en Hobbiton y 1 en Bree), 2 Magos (1 ubicado en Rivendell y otro en Isengard), 1 Elfo (ubicado en Lothlorien), 2 Enanos (ubicados en Fangorn y Erebor), 2 Humanos (uno en Edoras y otro en Bree), 3 Orcos (en Moria) y 2 Corsarios (en Dol Amroth). Todos ellos están inicialmente disponibles, menos el Humano que está en Edoras. Orcos, Humanos y Corsarios serían nuevos tipos de personaje. Partiendo de los códigos del Ejercicio 3a, se deben actualizar todas las acciones necesarias para extender el comportamiento a lo indicado en este ejercicio.

El objetivo de este ejercicio es:

- 1) Que el anillo sea destruido por la comunidad (al igual que en los Ejercicios 2 y 3a);
- 2) Que se cree un Uruk-Hai;
- 3) Que un humano esté en Bree.

Ejercicio 5: acciones con costes variables (1 punto)

En este ejercicio vamos a experimentar con la posibilidad de que las acciones puedan modificar variables numéricas o, en nuestro caso concreto, que las acciones puedan tener un coste no unitario. Para ello, vamos a retomar nuestro código del Ejercicio 1 (tanto el fichero de dominio como el de problema) y vamos a modificar únicamente lo necesario para introducir costes de acción no unitarios. Es decir, vamos a asumir que los costes para desplazarse por el mapa de la Tierra Media no son constantes/unitarios. En concreto, todos los desplazamientos entre ubicaciones son unitarios (cuestan 1) salvo los siguientes⁸:

Tharbad - Helm's Deep: 3
Minas Morgul - Orodruin: 3
Rivendell - High Pass: 2
Rivendell - Moria: 3
Lothlorien - Amon Hen: 5
High Pass - Mirkwood: 2
Mirkwood - Erebor: 2
Moria - Lothlorien: 3
Fangorn - Amon Hen: 8
Amon Hen - Dead Marshes: 2
Dead Marshes - Minas Morgul: 2
Minas Tirith - Minas Morgul: 2
Edoras - Minas Tirith: 2
Minas Tirith - Tolfolas: 2
Edoras - Dol Amroth: 2

⁸ El coste indicado afecta a ambas direcciones (ida y vuelta).



La definición general del ejercicio es idéntica a la del Ejercicio 1, por lo tanto los objetivos son también los mismos. Compare el plan obtenido en este ejercicio y en el Ejercicio 1, indicando las diferencias entre ambos y el motivo de las mismas.

Evaluación y Entrega

Los estudiantes deben entregar en la actividad creada en PRADO, **antes de las 23:59 del 02 de junio de 2026**, una carpeta comprimida en formato ZIP con el siguiente contenido:

- **Un fichero de dominio y otro fichero de problema para cada uno de los cinco ejercicios planteados. Dichos ficheros deben respetar los nombres “dominioX.pddl” y “problemaX.pddl”, donde “X” corresponde al número del ejercicio.** Es decir, por cada ejercicio debe haber un fichero de dominio y otro de problema, salvo que se indique lo contrario (véase Ejercicio 3).
- **Una breve memoria (10 páginas como máximo⁹)** donde se explique y justifique el diseño de la solución proporcionada (es decir, cuáles son las ideas clave que han guiado su implementación, tanto del fichero de problema como del fichero de dominio), así como se indique el uso que se ha realizado de IA generativa, y responda razonadamente a las preguntas que se hayan podido plantear.

Todos los ficheros deben estar adecuadamente **comentados, explicando y justificando la solución presentada, así como las variables, tipos y predicados usados para codificar cada problema, y cualquier otro aspecto relativo a las decisiones tomadas durante el proceso de implementación. Un código cuyos comentarios sean, a criterio del profesor, extremadamente deficientes podrá invalidar total o parcialmente aquellos ejercicios a los que afecte.**

De cara a considerar un ejercicio como correcto, el resultado debe ser correcto, el código debe estar bien comentado, y la memoria debe estar razonablemente completa y bien elaborada. Los problemas con **errores sintácticos** en PDDL, así como **código que no esté comentado en absoluto**, automáticamente califican con una puntuación de **0 (cero) puntos**.

⁹ Este límite de páginas, al igual que en las prácticas anteriores, es estricto e incluye todo: desde un posible índice, a una posible portada o sección bibliográfica.